=== UNIT 6: ASYNCHRONOUS JAVASCRIPT, PROMISES & FETCH API ===

>> 6.1 The JavaScript Event Loop & Asynchronous Runtime Model

JavaScript is fundamentally single-threaded: it possesses a single Call Stack and can execute only one instruction at a time. Concurrency is achieved through the browser runtime environment:
* Anatomy of the Concurrency Model:
- Call Stack: LIFO stack executing synchronous function frames.
- Web APIs: Browser-managed background multi-threaded threads handling timers (setTimeout), network requests (fetch), and DOM events.
- Callback (Task / Macrotask) Queue: Holds ready callbacks from setTimeout, setInterval, and I/O.
- Microtask Queue: High-priority queue holding resolved Promise callbacks (.then, .catch, async/await). Microtasks execute immediately upon Call Stack clearing, BEFORE the next macrotask.
- The Event Loop: Continuous background coordinator that constantly checks if the Call Stack is empty. If empty, it dequeues all Microtasks first, then processes one Macrotask, repeating indefinitely.

>> 6.2 The Evolution of Asynchronous Paradigms

* 1. Callbacks: Passing functions as arguments to execute upon completion. Drawback: 'Callback Hell' / 'Pyramid of Doom' (nested unreadable error handling).
* 2. Promises (ES6): An object representing the eventual completion (or failure) of an asynchronous operation and its resulting value:
- Three States: Pending (in progress), Fulfilled (resolved successfully), Rejected (failed with error).
- Chaining: promise.then(onFulfilled).catch(onRejected).finally(onComplete).
* 3. Async / Await (ES2017): Syntactic sugar over Promises enabling asynchronous code to read sequentially like synchronous code: async function fetchData() { const res = await fetch(...); }.

>> 6.3 JavaScript Object Notation (JSON)

JSON is the universal text-based data interchange format based on JavaScript object literal syntax:
* Syntax Rules: Keys must be double-quoted strings ('key'); strings in double quotes;

values can be string, number, object, array, boolean, null. No functions, comments, or undefined permitted.
* JSON Methods: JSON.stringify(obj, replacer, space) converts JS object to JSON string; JSON.parse(jsonString) parses JSON string into JS object.

>> 6.4 The Fetch API and RESTful HTTP Requests

The modern Promise-based native replacement for legacy XMLHttpRequest (XHR):
* Syntax: fetch(url, options). Returns a Promise resolving to a Response object.
* Two-Step Resolution: (1) fetch() resolves when HTTP headers are received: const response = await fetch(url). (2) Reading response body asynchronously: const data = await response.json().
* Checking HTTP Status: fetch() rejects ONLY on network failure (DNS error, offline). It resolves normally on HTTP 404 or 500 errors! Developers must explicitly check response.ok (true for status 200-299).

>> 6.5 Client-Side Web Storage API

* LocalStorage: Persistent key-value storage surviving browser restarts; 5-10 MB quota per origin: localStorage.setItem(k, v), .getItem(k), .removeItem(k), .clear().
* SessionStorage: Key-value storage cleared automatically when the specific browser tab is closed.
* Cookies: Sent automatically with every HTTP request; limited to 4 KB; managed via document.cookie.

[PROGRAM 1: Asynchronous Weather Dashboard via Fetch API]

Problem Context: Build a weather dashboard module that fetches live weather telemetry from a RESTful API using async/await, handles HTTP errors cleanly, and updates the DOM dynamically.
JavaScript Source Code:

```
// Asynchronous REST API Consumer with Error Handling
async function fetchWeatherData(cityName) {
const weatherDisplay = document.getElementById('weatherDisplay');
weatherDisplay.innerHTML = '<p class='loading'>Fetching latest satellite data...</p>';

const apiKey = 'DEMO_KEY_12601';
const url = 'https://api.weatherapi.com/v1/current.json?key=' + apiKey + '&q=' +
```

```
encodeURIComponent(cityName);

try {
const response = await fetch(url);

// Explicitly verify HTTP status code
if (!response.ok) {
throw new Error('HTTP Error: ' + response.status + ' (' + response.statusText + ')');
}

const data = await response.json();
renderWeatherUI(data);
} catch (err) {
console.error('Fetch Failed:', err);
weatherDisplay.innerHTML = '<p class='error'>Failed to load weather: ' + err.message +
'</p>';
}
}

function renderWeatherUI(data) {
const { location, current } = data;
let html = '<div class='weather-card'>';
html += '<h3>' + location.name + ', ' + location.country + '</h3>';
html += '<p class='temp'>' + current.temp_c + '&deg;C</p>';
html += '<p class='condition'>' + current.condition.text + ' | Humidity: ' +
current.humidity + '%</p>';
html += '</div>';
document.getElementById('weatherDisplay').innerHTML = html;
}
```

Key Features: encodeURIComponent sanitization, explicit response.ok verification, and try/catch block.

[PROGRAM 2: Promise Combinators: Promise.all and Promise.race]

Problem Context: Demonstrate parallel API fetching using Promise.all to fetch student grades, course catalog, and attendance concurrently in O(max(t)) time rather than serial O(sum(t)).

JavaScript Source Code:

```
const fetchProfile = () => new Promise(res => setTimeout(() => res({ name: 'Aarav' }),
300));
```

```
const fetchGrades = () => new Promise(res => setTimeout(() => res({ gpa: 9.2 }), 500));
const fetchAttendance = () => new Promise(res => setTimeout(() => res({ percent: 94 }),
200));

async function loadStudentDashboard() {
console.time('ParallelFetch');
try {
// Fetch all three endpoints concurrently
const [profile, grades, attendance] = await Promise.all([
fetchProfile()
fetchGrades()
fetchAttendance()
]);

console.timeEnd('ParallelFetch'); // Takes ~500ms (longest single promise), NOT 1000ms!
console.log('Aggregated Dashboard Data:', { ...profile, ...grades, ...attendance });
} catch (err) {
console.error('One of the parallel requests failed:', err);
}
}
loadStudentDashboard();
```

Key Takeaway: Promise.all executes operations concurrently, failing fast if any single promise rejects.

[PROGRAM 3: POST Request with JSON Payload and Headers]

Problem Context: Submit a user registration payload to a REST API via fetch POST method, sending JSON headers and handling the JSON response.

JavaScript Source Code:

```
async function submitRegistration(userData) {
try {
const response = await fetch('https://api.lpu.in/students/v1', {
method: 'POST'
headers: {
'Content-Type': 'application/json'
'Accept': 'application/json'
'Authorization': 'Bearer token_abc123'
}
body: JSON.stringify(userData) // Serialize object to JSON string
});
```

```
if (!response.ok) throw new Error('Registration failed with status
'${response.status}');
const result = await response.json();
console.log('Server Confirmation:', result);
} catch (err) {
console.error('Submission Error:', err.message);
}
}
```

[PROGRAM 4: LocalStorage Persistent Shopping Cart Manager]

Problem Context: Implement a persistent shopping cart using LocalStorage, supporting adding items, serializing array to JSON, updating quantities, and clearing cart.

JavaScript Source Code:

```
class CartManager {
constructor() {
this.storageKey = 'lpu_student_cart';
this.cart = this.loadCart();
}

loadCart() {
const stored = localStorage.getItem(this.storageKey);
return stored ? JSON.parse(stored) : [];
}

saveCart() {
localStorage.setItem(this.storageKey, JSON.stringify(this.cart));
}

addItem(id, name, price) {
const existing = this.cart.find(item => item.id === id);
if (existing) {
existing.qty += 1;
} else {
this.cart.push({ id, name, price, qty: 1 });
}
this.saveCart();
}
```

```
getTotal() {
return this.cart.reduce((sum, item) => sum + item.price * item.qty, 0);
}
}

const myCart = new CartManager();
myCart.addItem(101, 'Data Structures Textbook', 650);
console.log('Cart Total: INR', myCart.getTotal());
```

>> 6.6 University Review & Examination Questions

* Q1: Detail the internal mechanics of the JavaScript Event Loop, explaining the operational priority between the Microtask Queue and the Macrotask (Callback) Queue.
Answer: The JavaScript runtime contains a single Call Stack, Web APIs, a Callback (Macrotask) Queue, and a Microtask Queue. When asynchronous APIs (e.g. setTimeout) complete, their callbacks enter the Macrotask Queue. When Promises resolve (.then, .catch, await), their callbacks enter the Microtask Queue. The Event Loop continuously monitors the Call Stack. When the Call Stack empties, the Event Loop prioritizes the Microtask Queue, draining EVERY microtask until the queue is completely empty. Only after all microtasks are exhausted does the Event Loop take ONE macrotask from the Macrotask Queue and push it onto the Call Stack. This explains why resolved Promise callbacks execute before setTimeout(..., 0) callbacks.
* Q2: Explain why 'fetch()' does not reject on HTTP 404 or 500 status codes, and demonstrate how robust client applications verify response success.
Answer: In the Fetch API, a Promise rejects strictly when a network-level failure occurs that prevents the browser from completing the HTTP transaction (e.g. loss of internet connection, DNS resolution failure, CORS configuration rejection). When a server receives the request and returns an HTTP status code (even an error code like 404 Not Found or 500 Internal Server Error), the HTTP communication succeeded. Consequently, the fetch() Promise fulfills successfully, returning a Response object. Robust applications must check the boolean property 'response.ok' (which is true only for status codes 200-299) or inspect 'response.status'; if response.ok is false, the application must throw a manual error to route into the catch block.

>> 6.7 Key Takeaways and Revision Points

* The Event Loop drains the Microtask Queue (Promises) before the Macrotask Queue (setTimeout).
* Async/await provides clean sequential syntax over Promise chains.
* Fetch API requires checking response.ok; it resolves normally on 404/500 HTTP

errors.

* LocalStorage persists stringified JSON across browser sessions (5-10 MB limit).